

What to write down, what to let Claude find

Documenting a repo so Claude does the right thing unprompted — without micromanaging file references.

1 WHAT CLAUDE LOADS AUTOMATICALLY AT SESSION START

Auto-loaded, every session

- ✓ `CLAUDE.md` — project + global, full text
- ✓ Memory index & recalled memories
- ✓ A **git status snapshot** — filenames only
- ✓ The working-directory path

NOT loaded — read only on demand

- ✗ Contents of `BACKLOG.md`, `docs/`, `src/` ...
- ✗ Any recursive file tree / inventory
- ✗ Anything until a task makes Claude go look

2 THE DISTINCTION THAT DECIDES EVERYTHING

Findability — free.

Search tools (glob/grep/read) locate any file the moment a task needs it.

Never document a file just so Claude can find it — it always can.

Awareness of a convention — must be told.

A file's existence never reveals the **decision** behind it. "Completed work goes here, condensed, not in BACKLOG" is a choice, invisible in the file itself.

3 THE HEURISTIC

Document decisions, not contents.

The pointers you need are already hiding inside the decisions.

The test: Would a competent engineer, dropped in with search tools, figure it out from the code? **Yes** → don't write it (file layout, what a function does). **It's a choice invisible in any file** → write it (keep Singularity out of the hot path; finished work → CHANGELOG).

4 THREE KINDS OF REFERENCE

✓ **KEEP**

Byproduct of a convention

The link is welded to a behavior — you don't author it separately.

"finished work → CHANGELOG"

✓ **KEEP**

Routing index

"Read X **when** Y happens." A trigger — a convention in table form.

docs/INDEX.md's "read when" column

✗ **AVOID**

Bare inventory

"Here's every file." Redundant with search, and it rots into a confident lie.

a stale table of contents

Document decisions, not contents.

A ToC on top of that is a convenience for *you* — and the part most likely to go stale. Build it only if you'll use it.